Name : Aditi Rajak

PRN: 24070521113

Section: B

```
sit@sit-HP-ProOne-440-23-8-inch-G9-All-in-One-Desktop-PC:~/cc$ yacc -d practical11.y
sit@sit-HP-ProOne-440-23-8-inch-G9-All-in-One-Desktop-PC:~/cc$ flex practical11.l
sit@sit-HP-ProOne-440-23-8-inch-G9-All-in-One-Desktop-PC:~/cc$ gcc y.tab.c lex.yy.c -o practical11 -ll -lm
sit@sit-HP-ProOne-440-23-8-inch-G9-All-in-One-Desktop-PC:~/cc$ ./practical11
Enter an arithmetic expression: a+b*c
t1 = b * c
t2 = a + t1
t2 = a + t1

Intermediate code generated successfully.
Invalid expression: syntax error
sit@sit-HP-ProOne-440-23-8-inch-G9-All-in-One-Desktop-PC:~/cc$
```

Name : Aditi Rajak

PRN: 24070521113

Section: B

practical11.y

```
%{
#include "y.tab.h"
#include <stdlib.h>
#include <string.h>
%}

%%
[a-zA-Z][a-zA-Z0-9]*  { yylval.str = strdup(yytext); return ID; }
[0-9]+                { yylval.str = strdup(yytext); return NUM; }
[+\-*/()]             { return yytext[0]; }
[ \t\n]               ;
.                     { return yytext[0]; }
%%

int yywrap()
{
    return 1;
}

```

Name : Aditi Rajak

PRN: 24070521113

Section: B

*practical11.y

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex(void);
int yyerror(const char *s);

int temp = 1;

char* newtemp()
{
    char *t = malloc(20);
    sprintf(t, "t%d", temp++);
    return t;
}
%}

%union {
    char *str;
}

%token <str> ID NUM
%type <str> E

%left '+' '-'
%left '*' '/'
%right UMINUS

%%

S : E
    {
        printf("\nIntermediate code generated successfully.\n");
    }
  ;

E : E '+' E
    {
        $$ = newtemp();
        printf("%s = %s + %s\n", $$, $1, $3);
    }
  | E '-' E
    {
        $$ = newtemp();
        printf("%s = %s - %s\n", $$, $1, $3);
    }
  | E '*' E
```

Name : Aditi Rajak

PRN: 24070521113

Section: B

```
  | E '*' E
    {
        $$ = newtemp();
        printf("%s = %s * %s\n", $$, $1, $3);
    }
  | E '/' E
    {
        $$ = newtemp();
        printf("%s = %s / %s\n", $$, $1, $3);
    }
  | '-' E %prec UMINUS
    {
        $$ = newtemp();
        printf("%s = -%s\n", $$, $2);
    }
  | '(' E ')'
    {
        $$ = $2;
    }
  | ID
    {
        $$ = $1;
    }
  | NUM
    {
        $$ = $1;
    }
  ;

%%

int main()
{
    printf("Enter an arithmetic expression: ");
    yyparse();
    return 0;
}

int yyerror(const char *s)
{
    printf("Invalid expression: %s\n", s);
    return 0;
}
```